

CS837: ADVANCED OPERATING SYSTEMS

Operating System Overview & Core Foundations

Dr. Sarosh Tahir

sarosh.tahir@seecs.edu.pk

School of Electrical Engineering and Computer Science (SEECS)

National University of Sciences and Technology (NUST)

Lectures combine slides and board discussions to reinforce key OS concepts

Administration & Philosophy

- **Interplay of Methods:** Classes will actively combine **slides and board work** to illustrate theoretical concepts and mathematical proofs.
- **LMS Accessibility:** Core lecture slides will be posted regularly on the LMS platform.
- **No Scribe Policy:** Scribes are not officially provided. Students are expected to **take their own notes** during active board discussions.

Slide Updates & Pre-requisites

- **Dynamic Updates:** Slides are **living documents**. The instructor may update or correct them at any stage of the semester.
- **Best Practice:** Always download the latest version of the slides from LMS immediately prior to each class.
- **Quotes & Inspiration:** *I hear and I forget. I see and I remember. I do and I understand.* — Confucius

Operating systems manage shared hardware resources and bridge the user-to-hardware gap

"Operating systems are those programs that interface the machine with the applications programs. The main function of these systems is to dynamically allocate the shared system resources to the executing programs... concerned with the management and scheduling of memory, processes, and other devices..."

— WHAT CAN BE AUTOMATED?: THE COMPUTER SCIENCE AND ENGINEERING RESEARCH STUDY,
MIT Press, 1980

Key takeaway: An operating system is fundamentally both a software intermediary that abstracts hardware, and an allocator responsible for managing resource contention among competing applications.

Operating systems provide simplified abstractions and manage critical resources like memory

Three Core Pillars of an OS

- **Simplifying Intermediary:** Programs that make it fundamentally easier and more efficient to execute software applications.
- **Abstractions Provider:** Presents simple, elegant virtual abstractions of physical hardware, shielding programs from low-level register/timing complexities.
- **Resource Allocator:** Resolves conflict between multi-user resource demands, scheduling execution and managing storage.

OS Classification & Industry Examples

- **Desktop & Servers:**

Windows XP/2000/10/11, Ubuntu Linux, FreeBSD, Fedora, Solaris, macOS

- **Mobile Operating Systems:**

Android, iOS, Symbian OS, Lynx OS

- **Legacy Environments:**

DOS, IBM OS/2

Convenience, efficiency, and adaptability define the core objectives of modern operating systems

1. CONVENIENCE

- **User Centricity**

Makes computers more accessible and intuitive for human operators.

- **Developer Leverage**

Provides high-level APIs so programmers don't write device-specific driver codes.

2. EFFICIENCY

- **Asset Maximization**

Utilizes physical CPU cycles, RAM blocks, and storage blocks in an optimal, non-wasteful manner.

- **Throughput Scaling**

Manages process overhead to run more jobs per second.

3. ABILITY TO EVOLVE

- **Future Proofing**

Designed to seamlessly integrate new hardware features, modules, and upgrades without system disruption.

- **Continuous Improvement**

Allows runtime patching for security flaws, bug fixes, and new services.

The system infrastructure spans hardware, OS, libraries, and application software layers

Application Programs

Application Programming Interface (API) Boundary

Libraries & Utilities

Application Binary Interface (ABI) Boundary

Operating System

Instruction Set Architecture (ISA) Boundary

Execution Hardware

System Infrastructure Elements

- **Application Programs:** The topmost layer where processes run to solve user-specified domain tasks. Executes in protected user space.
- **Libraries & Utilities:** Translates source API commands into runtime-enforceable binary ABI instructions.
- **Operating System:** Intermediary kernel space software which direct operations of underlying CPUs, registers, and memories.
- **Execution Hardware:** Physical execution silicon, memory buses, caches, network cards, and secondary platters.

An operating system delivers seven vital services including program execution and file access

Development & Execution Services

- **Program Development:** Offers utility tools (linkers, editors, debuggers) to streamline software development. Often packaged with OS platforms.
- **Program Execution:** Automates program loading into memory, schedules hardware instruction blocks, and configures physical execution context.
- **Access I/O Devices:** Standardizes hardware calls so developers can use abstract commands, hiding physical device controller register complexity.
- **Controlled File Access:** Governs structured file storage, implements directory structures, and enforces device physical media mapping.

Protection & System Management

- **System Access:** Arbitrates shared resource access in multi-user networks, protecting storage partitions and enforcing user credentials.
- **Error Detection & Response:** Intercepts hardware-level faults (parity errors, device failures) and software-level crashes (arithmetic traps, page violations).
- **Accounting:** Dynamically monitors resource utilization metrics for performance profiling, capacity planning, and server billing.

Three key interfaces—ISA, ABI, and API—govern system-to-hardware communication

Defining Interface Boundaries

- **1. Instruction Set Architecture (ISA)**

Defines the literal machine language instruction repertoire (registers, opcodes, memory modes) supported by physical silicon.

- **2. Application Binary Interface (ABI)**

Establishes binary standards for program interaction across libraries and OS. Governs register conventions and runtime constraints.

- **3. Application Programming Interface (API)**

Defines source-level function calling signatures, data types, and interface contracts for high-level language software.

Interface Migration Dynamics

- **Boundary Shifting:** The boundaries between interfaces are fluid and shift over time based on research and performance trends.

- **Hardware Migration:** Frequently, functions that were originally purely implemented in OS software migrate directly into the hardware chip design.

- **Software Migration:** Extraneous user library codes often get directly absorbed into core operating system modules to improve efficiency.

APIs govern source code compatibility, while ABIs define the binary runtime interaction

API: Application Programming Interface

- **Source Code Contract:** Establishes standard interfaces for source-level code. Developers write programs targeting a specific API.
- **Build-Time Enforced:** Evaluated during compilation by the compiler which checks types and function signatures.
- **Hardware Independent:** Keeps high-level source portable across varying microarchitectures.

ABI: Application Binary Interface

- **Binary Link Contract:** Enforces rules governing compiled machine code compatibility.
- **Runtime Enforced:** Set by the linker and OS. Dictates register-passing rules, stack layouts, and system call opcodes.
- **Environment Bound:** Specific to a operating system kernel and physical CPU architecture (e.g. Linux x86_64 ABI).

The operating system controls system resources from within, unlike external control mechanisms

Traditional Control (External)

- **External Entity:** Traditional control structures sit entirely outside the system they control.
- **Passive Monitored:** Controls the system from an isolated environment, utilizing independent wiring.
- **Example: Heating & Thermostat:** A thermostat sits on the wall, completely independent of the furnace heating elements and boiler tanks.

OS Control (Internal Software)

- **Internal Soft Entity:** An OS is not external hardware; it is standard software executed by the processor.
- **Resource Competitor:** Shares memory, shares storage buses, and competes for CPU cycles on the same chip as user tasks.
- **Cooperative Preemption:** Relinquishes processor control to user programs and must depend on hardware interrupts to regain control.

The OS functions as standard software but frequently relinquishes control to user programs

Software Execution Profile

- **Executable Suite:** At its core, an OS is a program (or suite of programs) written in low-level languages (C, Assembly) and executed directly on the physical processor.
- **Memory Footprint:** Partially resident in main memory. Non-resident utilities are loaded from secondary storage on demand.
- **Privilege Boundary:** Executes instructions in high-privilege kernel mode while application processes execute in low-privilege user mode.

The Yield & Reclaim Mechanism

- **Control Relinquishment:** The OS does not run continuously. It starts a user process, sets hardware registers, and yields CPU execution.
- **Processor Dependence:** Once yielded, the OS depends entirely on processor hardware structures to regain execution control.
- **Interrupt Loop:** Hardware events (timer ticks, I/O completion, system traps) force the CPU to context-switch back into OS kernel handlers.

The operating system dynamically allocates CPU, memory, and I/O resources to active jobs

PROCESSOR (CPU) MANAGEMENT

- Schedules active processes.
- Manages context switches.
- Controls quantum slices to optimize CPU utilisation.

MAIN MEMORY ALLOCATION

- Maps virtual memory page structures.
- Allocates physical RAM block partitions.
- Controls page sharing and memory boundaries.

SECONDARY STORAGE CONTROL

- Organizes folders and file indices.
- Maps storage blocks on physical platters.
- Schedules disk access requests to minimize seek times.

I/O DEVICES ARBITRATION

- Handles buffering, caching, and queuing.
- Schedules device-level data transfer cycles.
- Arbitrates concurrent physical device demands.